A Project Report on

Fake News Detection and Classification Department of CSE

submitted in partial fulfillment for the award of

Bachelor of Technology

in

Computer Science & Engineering

by

N. Reethika (Y22ACS516) **P. Nitish Kumar (Y22ACS533)**

M. A.V. Naga Lakshmi (Y22ACS492) **Sk. Malik Babu(Y21ACS569)**

Under the guidance of

T. Nagarjuna, M.Tech(Ph.d)

Department of Computer Science and Engineering

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522102, Andhra Pradesh, INDIA

2025-2026

Department of
Computer Science & Engineering

CERTIFICATE

This is to certify that the project report entitled **Fake News Detection and Classification** that is being submitted by N Reethika (Y22ACS516), P Nitish Kumar (Y22ACS533), M A V Naga Lakshmi (Y22ACS492), Sk Malik Babu (Y21ACS569) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide
T. Nagarjuna, M.Tech(Ph.d)

Signature of the HOD
Dr. M. Rajesh Babu
Assoc. Prof. & Head

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

N Reethika (Y22ACS516)
P Nitish Kumar (Y22ACS533)
M A V Naga Lakshmi (Y22ACS492)
Sk Malik Babu (Y21ACS569)

Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Mr. T. Nagarjuna, M.Tech(Ph.d),** Department of CSE, for his valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. M. Rajesh Babu,** Associate Professor & Head of the Dept. for extending his cooperation and providing the required resources.

We would like to thank our beloved Principal **Dr. B. Srinivasa Rao** for providing the online resources and other facilities to carry out this work.

We express our sincere thanks to our Project Coordinators, **Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi**, **Assistant Professor, Department of CSE,** for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our parents and family members for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

We extend our sincere thanks to all other teaching faculty and non-teaching staff of the department, who helped directly or indirectly for their cooperation and encouragement.

N Reethika (Y22ACS516)
P Nitish Kumar (Y22ACS533)
M A V Naga Lakshmi(Y22ACS492)
Sk Malik Babu (Y21ACS569)

Table of Contents

Abstract viii

1 Introduction 1

1.1 Machine learning 1

1.1.1 Machine Learning vs Traditional Programming 3

1.1.2 How Does Machine Learning Work? 5

1.1.3 How to Choose Machine Learning? 7

1.2 Introduction to Neural Networks & Deep Learning 8

1.2.1 Understanding Deep Learning 8

1.2.2 Architecture of Neural Networks 9

1.2.3 Activation Functions 11

1.2.4 Deep Learning Models 11

1.3 Motivation 12

1.4 Problem Statement 13

1.5 Scope of the Project 14

2 Literature Survey 16

3 System Analysis 18

3.1 Existing System 19

3.2 Proposed System 27

3.2.1 Dataset Preparation and Preprocessing 29

3.2.2 Text Representation and Classification using BERT 30

3.2.3 Evidence Retrieval using External APIs 31

3.2.4 NLI-Based Verification using BART 32

3.2.5 Hybrid Scoring and Decision Making 33

3.2.6 Training Process 33

3.2.7 Evaluation 34

3.3 Advantages of the Proposed System 35

4 Implementation 37

4.1 Software Requirements 38

4.2 Hardware Requirements 38

4.3 Libraries and APIs Used 39

4.3.1 Flask 39

4.3.2 Transformers (HuggingFace) 40

4.3.3 Torch (PyTorch) 40

4.3.4 Scikit-learn 41

4.3.5 Pandas 42

4.3.6 NumPy 42

4.3.7 NewsAPI 43

4.3.8 Google Search API 43

4.3.9 FactCheck API 44

4.3.10 OS 44

4.4 Code 45

5 Results 49

6 Conclusion 54

7 Future Work 56

8 Bibliography 58

List of Figures

Figure 1.1 Architecture of neural network 10

Figure 3.1 Feature Based Detection 21

Figure 3.2 CNN Model 24

Figure 3.3 Proposed System work flow 28

Figure 5.1 User input 50

Abstract

The rapid growth of digital media and online platforms has significantly increased the spread of fake news, making it challenging to identify reliable information. Misinformation can influence public opinion, create confusion, and lead to serious social consequences. Therefore, there is a strong need for an efficient and automated system to detect and classify fake news.

This project presents a Fake News Detection and Classification system using advanced transformer-based models. The system employs BERT (Bidirectional Encoder Representations from Transformers) to analyse and classify news articles as real or fake based on deep contextual understanding of text. To further improve reliability, a BART-based Natural Language Inference (NLI) model is used to verify whether the collected external evidence supports or contradicts the given news claim.

The methodology includes comprehensive preprocessing of textual data, construction of labelled datasets, and the use of data augmentation techniques to improve model robustness. Training strategies and fine-tuning processes for both BERT and BART models are discussed to optimize performance in real-world scenarios.

Evaluation metrics such as accuracy, precision, recall, and F1 score are utilized to assess the effectiveness of the system in distinguishing between authentic and fabricated news. The abstract also highlights challenges such as handling ambiguous claims, dependency on external evidence quality, and computational complexity.

Keywords: BERT, BART, NLP, NLI, Flask, Misinformation.

1 Introduction

The exponential growth of digital media platforms has led to an unprecedented rise in the spread of misinformation and fake news. Fake news refers to deliberately misleading or false information presented as authentic news, often designed to manipulate public opinion, create confusion, or achieve political or financial gain. With the advancement of artificial intelligence and content generation tools, the creation and dissemination of fake news have become faster, more sophisticated, and harder to detect.

Traditional fact-checking approaches rely on manual verification by experts, which is time-consuming, labor-intensive, and not scalable to the massive volume of information generated daily. This has created an urgent need for automated, intelligent systems capable of detecting and classifying fake news in real time.

TruthLens PRO is a machine learning-based fake news detection and classification system designed to address these challenges. The system integrates supervised learning, deep learning models, and evidence-based reasoning to evaluate the authenticity of claims. It utilizes advanced transformer-based models such as BERT for text understanding and a BART-based Natural Language Inference (NLI) model for evidence verification. Additionally, it incorporates external APIs to retrieve credible evidence, enhancing decision-making accuracy.

The system classifies news into multiple categories including REAL, FAKE, and UNCERTAIN, using a hybrid scoring mechanism that combines model predictions and evidence validation. This multi-layered approach improves reliability and reduces the risk of incorrect classifications.

1.1 Machine learning

Machine learning is a branch of artificial intelligence that enables systems to automatically learn patterns from data and make predictions or decisions without being explicitly programmed. Instead of relying on fixed rules, machine learning models improve their performance over time by learning from historical data. This

makes them highly effective for solving complex problems where manual rule creation is difficult or impractical.

In the context of **fake news detection**, machine learning plays a crucial role in analyzing textual data, identifying linguistic patterns, and understanding contextual relationships within news content. It helps in distinguishing between authentic and misleading information by learning from previously labelled examples. Features such as word usage, sentence structure, sentiment, and contextual meaning are used to determine whether a claim is real or fake.

TruthLens PRO primarily employs **supervised machine learning**, where the model is trained on a labelled dataset containing examples of real and fake news. During the training phase, the model learns the relationship between input text and its corresponding label. Once trained, the model can generalize this knowledge to classify new, unseen claims with a high degree of accuracy.

In addition to basic classification, the system incorporates advanced techniques such as contextual embedding and evidence-based validation to enhance performance. The integration of deep learning models further improves the system's ability to understand nuanced language and detect subtle misinformation patterns.

The system supports:

Binary Classification: In this approach, the model classifies each claim into one of two categories: REAL or FAKE. It is a simple and efficient method that helps in making quick decisions based on the input text.

This type of classification is useful for basic fake news detection systems where only a clear distinction is required. However, it may not handle complex cases where the information is partially correct or unclear.

Multi-Class Classification: To provide more detailed results, the system extends binary classification into multiple categories such as REAL, FAKE, and UNCERTAIN. This allows better handling of ambiguous or partially correct information.

It improves the system's flexibility by considering different levels of truth, making the classification more informative and reliable compared to simple binary outputs.

Handling Imbalanced Data: In real-world datasets, one class (such as real news) may dominate the others, leading to imbalanced data. This can affect the model's ability to correctly identify minority classes like fake news.

To address this, techniques such as threshold tuning and balanced prediction strategies are used, ensuring fair and accurate classification across all categories.

Handling Uncertainty: The system is designed to handle uncertain or conflicting information effectively. Instead of forcing a decision, it assigns an UNCERTAIN label when there is not enough evidence.

This approach reduces the chances of incorrect predictions and increases the overall trustworthiness and reliability of the system.

Role of Machine Learning: Machine learning forms the core of the TruthLens PRO system by enabling automated fake news detection. It helps the system learn patterns from data and improve its performance over time.

With continuous learning and adaptation, the system can handle evolving misinformation trends and provide accurate, scalable, and efficient solutions in real-world scenarios.

1.1.1 Machine Learning vs Traditional Programming

Traditional programming and machine learning represent two fundamentally different approaches to problem-solving in computer systems.

In **traditional programming**, developers explicitly define a set of rules and logical instructions to process input data and produce the desired output. The system strictly follows these predefined rules, and any change in behavior requires manual modification of the code. This approach works well for problems with clearly defined logic and predictable outcomes, such as mathematical calculations or database operations.

However, in complex domains like **fake news detection**, traditional programming becomes ineffective. Human language is highly unstructured, ambiguous, and context-dependent. It is nearly impossible to write a comprehensive set of rules that can capture all variations of misleading content, sarcasm, tone, and contextual nuances. Additionally, misinformation evolves rapidly, making static rule-based systems outdated and unreliable over time.

In contrast, **machine learning** takes a data-driven approach. Instead of manually coding rules, the system learns patterns directly from data. By training on large datasets containing labelled examples (e.g., real and fake news), machine learning models automatically identify relationships between input features and output labels. This enables the system to generalize its learning and make accurate predictions on unseen data.

In the case of TruthLens PRO, machine learning models analyze linguistic features such as word choice, sentence structure, semantic meaning, and contextual relationships. Advanced models like BERT further enhance this capability by understanding bidirectional context within text, allowing the system to capture subtle differences between authentic and misleading information.

Another key advantage of machine learning over traditional programming is **adaptability**. Machine learning models can be retrained or fine-tuned with new data, allowing them to stay up-to-date with evolving misinformation trends. This makes them more robust and scalable compared to static rule-based systems.

Furthermore, machine learning supports **probabilistic decision-making**, where the model outputs confidence scores instead of rigid yes/no answers. This is particularly useful in fake news detection, where certainty may vary depending on available evidence. TruthLens PRO leverages this by combining model predictions with evidence-based scoring to produce more reliable results.

Despite its advantages, machine learning also introduces challenges such as the need for large labelled datasets, computational resources, and careful model evaluation. However, for complex and dynamic problems like misinformation detection, it remains the most effective and practical approach.

1.1.2 How Does Machine Learning Work?

Machine learning systems operate through a structured pipeline that involves data collection, preprocessing, model training, evaluation, and prediction. The primary goal is to enable the system to learn patterns from historical data and apply that knowledge to make accurate predictions on new, unseen inputs.

In the context of fake news detection, this process becomes more sophisticated as it involves understanding natural language, verifying facts, and handling uncertainty. TruthLens PRO follows a multi-stage workflow that integrates deep learning and evidence-based reasoning to ensure reliable classification

Workflow in TruthLens PRO:

Input Processing: The system begins by receiving a news claim, headline, or full article as input. This raw textual data may contain noise such as punctuation, irrelevant symbols, or inconsistent formatting. Initial preprocessing steps such as cleaning, normalization, and tokenization are applied to prepare the text for analysis.

Feature Extraction: Once the input is preprocessed, it is passed to the BERT model for feature extraction. BERT converts the text into contextual embeddings, which are numerical representations that capture the meaning of words in relation to their surrounding context. Unlike traditional methods, BERT understands the bidirectional relationship between words, making it highly effective for capturing nuanced language patterns.

Model Training and Prediction: During the training phase, the model learns from labelled datasets containing examples of real and fake news. It identifies patterns and relationships between input features and their corresponding labels.

During inference (prediction phase):

- The trained model processes the input embeddings.
- It generates probability scores indicating how likely the claim is real or fake.
- These scores form the **BERT prior (bert_prior)**.

Evidence Retrieval: To improve reliability, TruthLens PRO does not rely solely on model predictions. It integrates external Evidence Retrieval APIs to fetch relevant articles, fact-checking sources, or trusted news content related to the claim.

Evidence Selection and Similarity Matching: From the retrieved articles, the system identifies the most relevant sentences or passages. This is done using similarity techniques such as:

a. Token-based similarity
b. Character-level similarity

NLI (Natural Language Inference) Evaluation: The selected evidence is then evaluated using a BART-based NLI model. The model analyzes the relationship between the claim and each piece of evidence and classifies it into one of three categories:

a. **Entailment:** Evidence supports the claim
b. **Contradiction:** Evidence refutes the claim
c. **Neutral:** Evidence is unrelated or insufficient

Each result is assigned a score, which contributes to the overall **evidence_score**.

Hybrid Scoring Mechanism: To produce a balanced and reliable prediction, TruthLens PRO combines the BERT model's output with the evidence-based score using a weighted formula:

Hybrid Score = (0.45 × BERT Score) + (0.55 × Evidence Score)

This approach ensures that:

- Model predictions are considered
- Real-world evidence has slightly higher importance
- Final decisions are more trustworthy

Decision Thresholding and Classification: The hybrid score is compared against predefined thresholds to determine the final classification. Based on the score and evidence consistency, the system assigns one of the following labels:

- REAL
- FAKE
- UNCERTAIN

Final Output: The system provides a comprehensive output that includes the final classification label, a confidence score indicating the certainty of the prediction, and supporting evidence retrieved from external sources. This combined output helps users clearly understand the decision made by the system and increases transparency and trust in the results.

The System Outputs:

- Final classification label
- Confidence score
- Supporting evidence

1.1.3 How to Choose Machine Learning?

TruthLens PRO uses supervised learning combined with deep learning because of its high performance in natural language processing tasks and classification problems. In addition to these advantages, machine learning models can continuously improve as more data becomes available, enabling the system to stay updated with new types of misinformation.

Furthermore, the integration of transformer-based models such as BERT allows the system to understand contextual meaning beyond simple keywords, while models like BART enhance reasoning through evidence-based verification. This combination significantly improves the system's ability to detect subtle differences between real and fake news.

Machine learning is chosen for fake news detection due to its ability to:

Handle large-scale data efficiently: Machine learning models can process and analyze large volumes of data quickly, making them suitable for handling the vast amount of news content generated daily.

Adapt to evolving misinformation patterns: These models can learn from new data and update their understanding, allowing them to adapt to changing patterns of misinformation over time.

Capture complex linguistic and contextual relationships: Machine learning techniques, especially deep learning models, can understand complex language structures and contextual meanings within text.

Reduce dependency on manual verification: By automating the detection process, machine learning reduces the need for human intervention, making fake news detection faster and more scalable.

Overall, machine learning provides a scalable, adaptive, and intelligent solution for fake news detection, making it highly suitable for real-world applications like TruthLens PRO.

1.2 Introduction to Neural Networks & Deep Learning

Neural networks are computational models inspired by the structure and functioning of the human brain, consisting of interconnected layers of artificial neurons. These neurons process input data, apply transformations, and pass the results through the network to produce meaningful outputs.

Neural networks are capable of learning complex patterns, relationships, and representations from large datasets. In modern artificial intelligence, they form the foundation of deep learning systems used in tasks such as image recognition, speech processing, and natural language understanding.

In **TruthLens PRO**, neural networks play a crucial role in analyzing textual data, capturing semantic meaning, and identifying patterns associated with misinformation. This enables the system to accurately classify news content and detect fake or misleading claims.

1.2.1 Understanding Deep Learning

Deep learning is a specialized subset of machine learning that uses multi-layered neural networks to model complex patterns in data. These networks automatically learn hierarchical representations, where lower layers capture basic features and deeper layers capture more abstract concepts.

Deep learning is particularly powerful in **Natural Language Processing (NLP)**, where understanding context, grammar, and semantics is essential.

Transformer-based models have significantly improved the ability to process language efficiently and accurately.

TruthLens PRO leverages deep learning to:

- Understand contextual meaning of text beyond individual words
- Capture relationships between words and sentences
- Detect subtle linguistic differences between real and fake news
- Handle complex and ambiguous claims effectively
- Improve prediction accuracy through continuous learning and fine-tuning
- Integrate multiple data sources for more reliable and evidence-based decision-making

This enables the system to move beyond simple keyword matching and perform intelligent content analysis by understanding the context, meaning, and relationships within the text, leading to more accurate and reliable fake news detection.

1.2.2 Architecture of Neural Networks

A neural network is a computational model inspired by the human brain, designed to learn patterns and relationships from data. It consists of multiple interconnected layers that process input information step by step, enabling the system to perform tasks such as classification, prediction, and pattern recognition.

Input Layer: The input layer is the first layer of the neural network, which receives raw input data such as text, images, or numerical features. It acts as the entry point where data is fed into the network for further processing.

Hidden Layers: Hidden layers are intermediate layers that perform feature extraction, transformation, and learning of complex patterns. These layers apply mathematical operations and activation functions to identify important relationships within the data.

Output Layer: The output layer is the final layer of the network that produces the prediction or classification result. It provides the final decision, such as categorizing news as real or fake based on the learned patterns.

Each neuron processes data using weights, biases, and activation functions. During training, these parameters are updated using optimization techniques such as backpropagation to minimize prediction error.

However, in modern Natural Language Processing systems like TruthLens PRO, traditional neural networks are replaced by transformer-based architectures such as BERT and BART. These models use self-attention mechanisms to analyze entire sentences simultaneously rather than sequentially.

This allows the system to:

- Focus on important words in a sentence
- Capture contextual relationships more effectively
- Understand semantic meaning at a deeper level

As a result, transformer architectures significantly improve performance in tasks such as text classification, semantic analysis, and fake news detection, making them highly suitable for this project.

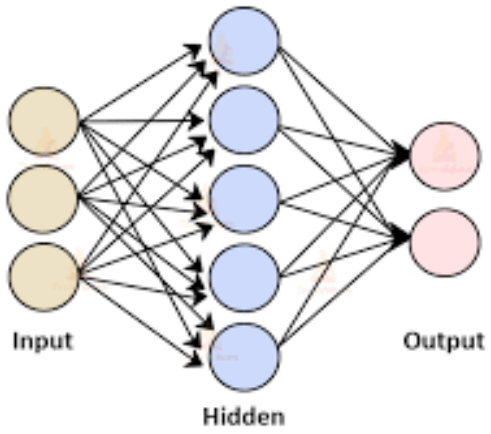


Figure 1.1 Architecture of neural network

1.2.3 Activation Functions

Activation functions are mathematical operations applied to neuron outputs, introducing non-linearity into the network. This enables neural networks to learn complex and non-linear relationships within data.

Common activation functions include:

- **ReLU (Rectified Linear Unit):** Widely used in hidden layers for efficient computation and faster training
- **Sigmoid:** Used in binary classification tasks to produce outputs between 0 and 1
- **Softmax:** Used in multi-class classification to generate probability distributions across classes

In transformer-based models like BERT, activation functions such as ReLU and GELU (Gaussian Error Linear Unit) are used internally within layers to improve learning efficiency.

In TruthLens PRO, the Softmax function is used in the final classification layer to assign probability scores to each category (REAL, FAKE, UNCERTAIN). The class with the highest probability is selected as the final prediction.

1.2.4 Deep Learning Models

TruthLens PRO utilizes advanced deep learning models to enhance the accuracy and reliability of fake news detection by combining contextual understanding with evidence-based validation. These models work together to analyze textual data, capture semantic relationships, and verify claims using real-world information, resulting in more trustworthy predictions.

BERT Model: BERT is a transformer-based model that processes text in both directions, which helps it understand the context of words more effectively than traditional models. Instead of reading text in a single direction, it looks at the entire sentence at once, making it better at capturing the actual meaning of the text.

It generates contextual embeddings for the input text, captures relationships between words in a sentence, and produces an initial classification probability

(bert_prior). This helps the system make an accurate first prediction about whether the given news is real or fake.

BART-based NLI Model: The NLI (Natural Language Inference) model evaluates the relationship between a given claim and the supporting evidence. It analyzes both the claim and the retrieved information to understand how they are connected and whether the evidence supports the claim or not.

It determines whether the evidence entails, contradicts, or is neutral with respect to the claim, and assigns scores based on their semantic alignment. This process helps in validating the claim using real-world information, improving the accuracy and reliability of the system's final decision.

Evidence Retrieval API: The system retrieves relevant articles and sources from external APIs to support the verification process. By collecting real-time information from trusted platforms, it ensures that the analysis is not limited to the model's internal knowledge.

This provides real-world context for verification, enhances decision-making beyond model predictions, and improves the overall reliability and trustworthiness of the system by validating claims with up-to-date external evidence.

Hybrid Scoring System: TruthLens PRO combines model predictions with evidence-based scoring to produce a more reliable final result. It uses a hybrid formula where the BERT score and the evidence score are weighted and combined, giving slightly more importance to real-world verification.

This approach balances learned patterns with factual verification, reduces false predictions, and improves overall classification accuracy. By considering both model intelligence and external evidence, the system provides more trustworthy and accurate outcomes.

1.3 Motivation

The rapid growth of digital platforms and social media has led to a significant increase in the spread of fake news and misinformation, creating serious concerns among individuals and organizations. Information can now be shared instantly across

the globe, making it easier for false or misleading content to reach a large audience within a short time.

Fake news has the potential to mislead people, influence public opinion, and create confusion in society. It can affect important areas such as politics, healthcare, and education, ultimately reducing trust in reliable and authentic information sources.

Traditional fact-checking methods are often slow and rely heavily on manual verification, which makes them unsuitable for handling the massive volume of data generated daily. This highlights the need for an automated and intelligent system capable of analyzing and verifying information quickly and accurately.

Our motivation is to develop an efficient and reliable fake news detection system, TruthLens PRO, using advanced deep learning models such as BERT and BART-based NLI, along with real-time evidence retrieval through APIs like NewsAPI, FactCheck API, and Google API. By combining machine learning with evidence-based verification, the system aims to improve accuracy, enhance trust, and help users easily identify credible information.

With the increasing complexity of misinformation, there is a growing need for systems that not only analyze text but also understand context and verify claims using reliable sources. Advanced Natural Language Processing techniques enable systems like TruthLens PRO to go beyond surface-level analysis and capture deeper semantic meaning, making detection more accurate and effective.

Furthermore, providing transparent and evidence-based results is essential to build user trust. By presenting supporting evidence along with predictions, the system ensures that users can understand and verify the decision-making process. This approach not only improves reliability but also empowers users to make informed judgments about the authenticity of information.

1.4 Problem Statement

In today's digital world, fake news is spreading rapidly across social media platforms and online sources, making it increasingly difficult for users to identify authentic information. Many news articles and claims appear genuine but are actually misleading or false, which creates confusion and misinformation among the public.

The widespread presence of fake news not only misleads individuals but also affects public opinion, decision-making, and trust in media sources. This growing issue highlights the need for reliable methods to verify the authenticity of information in a fast and efficient manner.

Currently, manual fact-checking methods are time-consuming, labor-intensive, and not scalable for handling the massive amount of data generated every day. This makes it difficult to detect and control the spread of misinformation in real-time.

To address these challenges, our project aims to develop an automated fake news detection system, TruthLens PRO, using advanced technologies such as BERT for text analysis and BART-based NLI for evidence verification. By combining machine learning with real-world evidence, the system helps users identify trustworthy information and reduces the spread of misinformation.

In addition to automation, the system focuses on improving the accuracy and reliability of predictions by combining contextual understanding with evidence-based verification. This ensures that the results are not only based on learned patterns but also supported by real-world information, making the system more dependable.

Furthermore, the proposed system is designed to be scalable and adaptable to future advancements. It can be extended to support multilingual data, real-time analysis, and integration with various online platforms, thereby increasing its effectiveness in combating misinformation on a larger scale.

1.5 Scope of the Project

The scope of this project focuses on developing a robust and efficient fake news detection system, TruthLens PRO, using machine learning and deep learning techniques. The system is designed to analyze textual data from news claims and classify them into categories such as REAL, FAKE, and UNCERTAIN, providing users with clear and reliable results.

The project emphasizes the use of advanced transformer-based models such as BERT for contextual understanding and BART-based NLI for evaluating the relationship between claims and supporting evidence. These models help in capturing semantic meaning and improving the overall accuracy of the classification process.

In addition to model-based predictions, the system integrates external APIs to retrieve real-world evidence related to the input claim. This enhances the decision-making process by combining machine learning outputs with factual verification, making the system more trustworthy and reliable.

The implementation includes dataset collection, preprocessing, model training, and evaluation. Performance is measured using metrics such as accuracy, precision, recall, and F1-score to ensure the effectiveness of the system in detecting fake news.

The project also covers system design, development, testing, and documentation. Future scope includes real-time deployment, multilingual support, and integration with social media platforms, which can further enhance the system's usability and impact in combating misinformation.

.

2 Literature Survey

The rapid increase in the spread of fake news has attracted significant research attention toward developing effective detection methods. Various approaches have been proposed in the literature, ranging from traditional machine learning techniques to advanced deep learning models.

Early research in fake news detection primarily focused on feature-based approaches, where handcrafted features such as lexical patterns, syntactic structures, and semantic cues were extracted from text data. These features were then used with classical machine learning algorithms to classify news as real or fake. While these methods showed moderate success, they often struggled to capture complex contextual relationships present in natural language.

With advancements in deep learning, Convolutional Neural Networks (CNNs) have been widely used for fake news detection. CNN-based models are effective in extracting local features and patterns from textual data, helping identify key phrases and structures associated with misinformation. However, CNNs have limitations in capturing long-range dependencies and deeper contextual meaning within text.

To overcome these limitations, researchers explored more advanced architectures capable of understanding context more effectively. Recent studies have highlighted the use of transformer-based models, which leverage attention mechanisms to process entire sentences and capture bidirectional context. These models significantly improve performance in natural language processing tasks.

Another important direction in fake news detection is evidence-based verification, where systems retrieve external information to validate claims. Instead of relying solely on textual features, these approaches incorporate real-world evidence to improve reliability and reduce false predictions. Natural Language Inference (NLI) models are commonly used in this context to determine whether retrieved evidence supports or contradicts a claim.

Despite these advancements, existing methods face challenges such as dependency on data quality, lack of explainability, and difficulty in handling

ambiguous or partially true claims. Additionally, many systems rely only on text classification without incorporating real-time fact-checking mechanisms.

To address these limitations, our proposed system, TruthLens PRO, integrates multiple advanced techniques. It utilizes a BERT-based model for accurate text classification by capturing contextual and semantic information. Additionally, a BART-based NLI model is employed to perform evidence verification by analyzing the relationship between claims and retrieved information. Furthermore, the system incorporates external APIs such as NewsAPI, FactCheck API, and Google API to obtain real-time evidence for fact-checking.

By combining deep learning with evidence-based reasoning and real-world data retrieval, the proposed system aims to provide a more robust, accurate, and reliable solution for fake news detection. This approach overcomes the limitations of traditional methods and enhances the overall effectiveness of misinformation detection systems.

3 System Analysis

In this chapter, we examine the existing methods used for fake news detection and analyze their working principles, strengths, and limitations. Understanding these systems helps in identifying the gaps and motivates the development of a more advanced and reliable solution.

Initially, fake news detection systems were based on traditional machine learning approaches that relied heavily on manually engineered features. These methods used techniques such as keyword extraction, linguistic patterns, and statistical features to classify news. While they were simple and computationally efficient, they often failed to capture deeper semantic meaning and context within the text.

With the advancement of deep learning, models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) were introduced for automatic feature extraction and sequence modelling. These models improved performance by learning patterns directly from data, enabling better detection of complex structures and relationships within text. However, they still had limitations in understanding long-range dependencies and contextual nuances.

Despite these improvements, existing systems struggle with challenges such as handling ambiguous language, detecting subtle misinformation, and verifying claims using real-world facts. These limitations highlight the need for more advanced approaches, such as transformer-based models and evidence-based verification systems, which can provide more accurate and reliable fake news detection.

Another major limitation of existing systems is their inability to integrate external knowledge sources effectively. Most traditional and deep learning models rely only on the training dataset and do not verify information using real-world evidence, which reduces their reliability in detecting factual inaccuracies.

To overcome these challenges, modern approaches focus on combining transformer-based models with evidence retrieval techniques. By integrating contextual understanding with real-time fact verification, these systems can provide more accurate, transparent, and trustworthy results, paving the way for advanced solutions like TruthLens PRO.

3.1 Existing System

The existing systems for fake news detection mainly use feature-based detection approaches and Convolutional Neural Networks (CNNs). These methods focus on analyzing textual patterns and extracting useful information from news content to classify it as real or fake, relying on statistical and linguistic features.

Feature-based approaches depend on manually engineered features such as word frequency, syntax, and writing style, while CNN-based models automatically learn patterns from the data through deep learning. These techniques have improved detection accuracy by capturing important textual characteristics and hidden patterns within the content.

However, these approaches have limitations in understanding deeper context, handling complex language, and verifying claims using external knowledge sources. As a result, they may fail to detect subtle misinformation, highlighting the need for more advanced models that combine contextual understanding with real-world evidence.

Feature-Based Detection Approach: Feature-based approaches are traditional methods that rely on extracting meaningful features from text data. These features are manually designed using domain knowledge, such as word frequency, sentence structure, and linguistic patterns, to represent the characteristics of the text.

Once these features are extracted, they are used to train machine learning models for classification tasks, such as identifying whether a news article is real or fake. Techniques like TF-IDF and Bag of Words are commonly used to convert text into numerical form for model processing.

While these methods are effective to some extent, they depend heavily on the quality of feature selection and manual effort. As a result, they often struggle to capture deeper semantic meaning and contextual relationships, limiting their performance in complex fake news detection scenarios.

Feature-based approaches are simple, interpretable, and require less computational power compared to deep learning models, making them easier to implement and understand. However, their reliance on handcrafted features limits

their scalability and adaptability, especially when dealing with large and complex datasets where automatic feature learning methods perform better.

Methodology: The process begins with data collection, where news articles or claims are gathered from various reliable and unreliable sources to create a diverse dataset. This is followed by preprocessing, where the text is cleaned by removing stopwords, punctuation, and unnecessary noise, making it suitable for further analysis.

Next, feature extraction is performed to identify important characteristics of the text. These include lexical features such as word frequency and n-grams, syntactic features like sentence structure and grammar, semantic features related to meaning and context, and stylistic features that capture writing style and tone.

Finally, the processed features are used for model training, where machine learning algorithms such as Logistic Regression, Naive Bayes, or Support Vector Machines (SVM) are applied to learn patterns from the data and classify news as real or fake.

Detection Mechanism: The detection mechanism begins with text preprocessing, where the input text is cleaned by removing stopwords, punctuation, and unnecessary symbols to eliminate noise. The text is then normalized and tokenized, converting it into a structured format suitable for further analysis and processing.

In the next step, feature engineering is performed to extract important characteristics from the text. These include lexical, syntactic, and semantic patterns, which are designed based on predefined rules to effectively distinguish between fake and real news.

After feature extraction, the data undergoes vectorization, where the extracted features are converted into numerical format using techniques such as TF-IDF or Bag of Words. This transformation enables machine learning models to efficiently process and analyze the textual data.

The classification stage involves applying machine learning algorithms that learn patterns and relationships from the numerical features during training. These models use the learned information to make predictions on new input data.

Based on these learned patterns, the system predicts whether the input news is REAL or FAKE, enabling automated and efficient fake news detection with improved accuracy and consistency.

Conclusion: Feature-based methods are simple and easy to interpret, as they rely on manually designed features based on domain knowledge. However, they depend heavily on the quality of feature selection and are less effective in capturing deep contextual meaning, making them less suitable for handling complex or evolving misinformation.

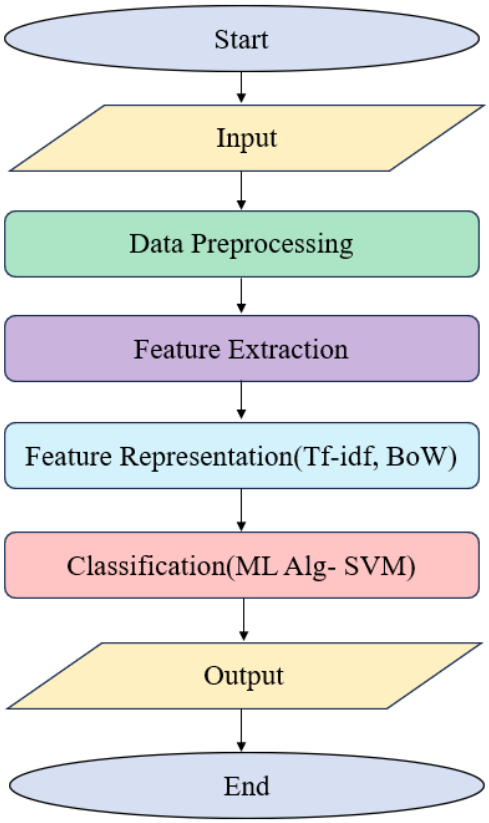


Figure 3.1 Feature Based Detection

Diagram Explanation: This diagram represents the overall workflow of a system, starting from the initial input stage and progressing through different processing steps until the final output is generated. The top oval shape indicates the start of the

process, where the system begins execution after receiving input from the user or an external source.

As the process moves forward, the system performs a series of operations such as data handling, processing, and intermediate decision steps. These stages involve analyzing the input, transforming it into a suitable format, and applying necessary logic or models to extract meaningful information. Each block in the diagram represents a specific function that contributes to the overall processing pipeline.

Finally, the system reaches the **end stage**, where the processed result is generated and displayed. This output represents the final decision or classification made by the system based on the input and processing steps. The diagram overall shows a structured flow from input to output, ensuring systematic and efficient execution of the task.

CNN Model: Convolutional Neural Networks (CNNs) are deep learning models that have been widely adapted for text classification tasks, including fake news detection. They are capable of automatically learning patterns from data, which makes them effective in identifying important features within textual content.

Unlike traditional feature-based methods, CNNs do not rely on manual feature engineering. Instead, they use convolutional layers to automatically extract meaningful features such as key phrases and local patterns from the input text, improving the efficiency and performance of the model.

CNNs are particularly useful in capturing hidden relationships and structural patterns within the text. By applying multiple filters, the model can detect various n-gram features and important signals that help in distinguishing between real and fake news.

However, CNNs may still face limitations in understanding long-range dependencies and deeper contextual meaning. While they are effective for local feature extraction, they may struggle with complex semantic relationships, which can affect their performance in advanced fake news detection scenarios.

CNN Methodology: In CNN-based fake news detection, the input text is first converted into word embeddings such as Word2Vec or GloVe, which represent words

in a numerical format. This transformation helps capture semantic relationships between words and prepares the text for further processing by the neural network.

After embedding, convolutional filters slide over the text to extract important patterns and features. Different filter sizes are used to capture various n-gram features, such as phrases and short sequences of words, which are crucial for understanding textual patterns.

The convolution operations generate feature maps, which are then passed through pooling layers. Pooling reduces the dimensionality of the data while retaining the most significant features, making the model more efficient and less prone to overfitting.

CNNs are particularly effective in identifying key phrases, repeated patterns, and local dependencies within the text. This ability allows the model to focus on important textual cues, helping improve the accuracy of fake news classification.

Detection Mechanism: The detection mechanism in CNN-based fake news detection begins with an embedding layer, where the input text is converted into dense vector representations using word embeddings such as Word2Vec or GloVe. This step captures semantic relationships between words and transforms the text into a numerical format suitable for neural network processing.

The next step is the convolution layer, where multiple filters are applied over the input embeddings to extract local patterns and important features. This helps the model identify key phrases, n-grams, and structural patterns associated with fake or real news.

After feature extraction, pooling layers such as max-pooling are used to reduce the dimensionality of the feature maps. This process retains the most important features while removing less relevant information, improving computational efficiency and reducing overfitting.

The extracted features are then passed to the fully connected layer, where they are combined and processed for final classification. The model learns to make decisions based on these features and generates probability scores for each class.

Finally, based on these probability scores, the system classifies the input news as REAL or FAKE. This structured approach allows CNNs to efficiently analyze textual patterns and improve the accuracy of fake news detection.

Conclusion: Convolutional Neural Networks (CNNs) provide an effective approach for fake news detection by automatically extracting important features such as key phrases and local patterns from text without the need for manual feature engineering. They are efficient in identifying meaningful structures within data and improving classification accuracy. However, CNNs have limitations in capturing long-range dependencies and deep contextual meaning, which can affect their performance in complex scenarios. Despite these challenges, CNNs serve as a strong foundation for text classification tasks and have significantly contributed to the advancement of fake news detection systems.

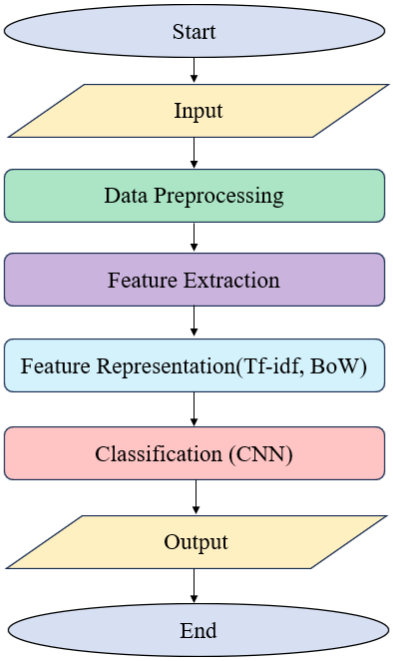


Figure 3.2 CNN Model

Diagram Explanation: This diagram represents a part of the fake news detection workflow, focusing on the stages after initial preprocessing. It highlights how the system processes text data through structured steps to prepare it for classification. The flow shows a clear progression from feature extraction to the final prediction stage.

In the first step, **Feature Extraction**, important patterns and characteristics are identified from the input text. These may include lexical, syntactic, and semantic features that help distinguish between real and fake news. The next stage is **Feature Representation**, where these extracted features are converted into numerical form using techniques such as TF-IDF or Bag of Words, making the data suitable for machine learning models.

Finally, the processed data is passed to the **Classification** stage, where machine learning algorithms analyze the numerical features and make predictions. Based on the learned patterns, the system classifies the input as REAL or FAKE. This structured pipeline ensures efficient processing and accurate decision-making in fake news detection.

Drawbacks of Feature-Based Approach

Requires manual feature engineering: Feature-based methods rely heavily on manually designing features using domain knowledge, which makes the process time-consuming and less scalable for large or diverse datasets.

Limited ability to capture semantic meaning: These approaches struggle to understand deeper context and meaning in text, as they focus mainly on surface-level features rather than true semantic relationships.

Poor adaptability to new data: Since the features are predefined, the model may not perform well on new or unseen data where patterns differ from the training dataset.

Dependent on domain knowledge: The effectiveness of the system largely depends on the expertise used to design features, which can limit generalization across different topics or domains.

Difficulty in handling complex language: Feature-based methods often struggle to handle complex language constructs such as sarcasm, irony, and implicit meanings, which are important for accurately detecting fake news.

Lack of automation: Feature-based methods do not automatically learn from data and require manual intervention at every stage, making the process less efficient compared to modern deep learning approaches.

Scalability issues: As the dataset size grows, designing and managing features becomes difficult, reducing the scalability of the system for real-world applications.

Feature redundancy: Manually created features may include irrelevant or redundant information, which can negatively impact model performance and increase complexity.

Limited performance improvement: Even after adding more features, performance improvements are often minimal, as the approach cannot capture deeper hidden patterns in data.

Drawbacks of CNN Models

Limited long-range dependency capture: CNN-based models focus on local patterns using small filters, which makes it difficult for them to capture long-range dependencies and relationships across distant words in a sentence.

Cannot fully understand context: Although CNNs can extract useful features, they lack the ability to fully understand contextual meaning, especially in complex or ambiguous sentences.

Requires large training data: CNN models typically require a large amount of labelled data to perform well, which can be challenging to obtain in fake news detection tasks.

No integration with external knowledge: These models rely only on the given dataset and do not incorporate real-world knowledge or external sources, limiting their ability to verify factual correctness.

Sensitive to input variations: CNN-based models can be sensitive to slight changes in input text, such as rephrasing or paraphrasing, which may affect their ability to consistently detect fake news accurately.

High computational cost: CNN models require significant computational resources, especially during training, which can increase time and cost for large-scale datasets.

Overfitting risk: CNNs may overfit the training data if not properly regularized, leading to poor generalization on unseen data.

Lack of interpretability: The decision-making process of CNNs is often not easily interpretable, making it difficult to understand why a particular prediction was made.

Difficulty in handling very long texts: CNNs may struggle when processing long documents, as important information located far apart may not be effectively captured.

3.2 Proposed System

Our proposed system, TruthLens PRO, is designed using advanced transformer-based models such as BERT and BART (NLI-based), combined with external evidence retrieval APIs. The system aims to detect and classify fake news by integrating deep semantic understanding with real-world fact verification, making it more reliable than traditional approaches.

Unlike conventional systems that rely primarily on surface-level textual features or keyword-based analysis, the proposed approach follows a multi-stage verification pipeline. In the first stage, the input claim is processed using BERT, which captures contextual relationships between words and generates meaningful representations of the text. This enables the system to understand not just individual words, but the overall intent and semantics of the claim.

In the second stage, the system retrieves relevant information from external sources such as NewsAPI, Google Search API, and fact-checking platforms. These sources provide real-time, reliable evidence that supports or contradicts the given claim. The retrieved evidence is then passed to a BART-based Natural Language Inference (NLI) model, which determines whether the evidence entails, contradicts, or is neutral with respect to the claim.

Furthermore, the system incorporates a hybrid scoring mechanism that combines the prediction from BERT with the evidence-based score generated by the NLI model. This weighted combination ensures that both learned patterns and factual verification contribute to the final decision, thereby reducing bias and improving overall classification accuracy.

The system is capable of categorizing news into multiple classes such as REAL, FAKE, and UNCERTAIN. This multi-class classification provides a more nuanced

understanding of information authenticity, especially in cases where claims are not entirely true or false.

Additionally, TruthLens PRO enhances transparency by presenting supporting and contradicting evidence to the user, enabling better interpretability of results. The use of transformer-based architectures also ensures scalability and adaptability, allowing the system to handle complex and ambiguous claims effectively.

Overall, this hybrid architecture significantly improves detection performance, minimizes false positives and false negatives, and enhances the trustworthiness of the system. It represents a modern and intelligent approach to combating misinformation in the digital era.

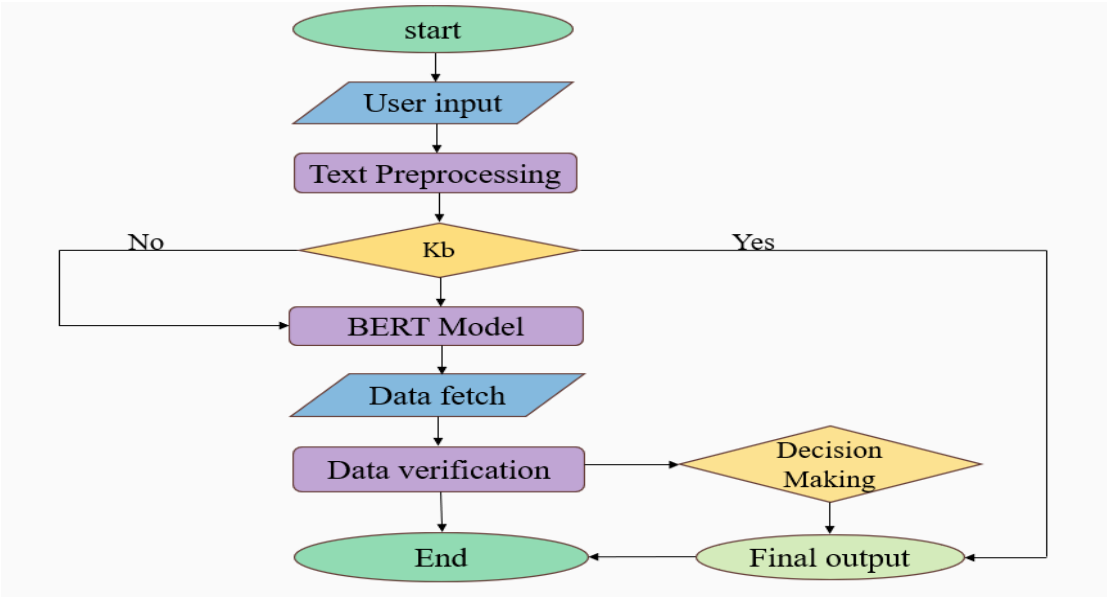


Figure 3.3 Proposed System work flow

Diagram Explanation: This diagram shows the overall working process of the fake news detection system. It starts with the user input, where a news article or claim is provided. The input text then goes through text preprocessing, where it is cleaned and prepared for further analysis by removing unwanted elements and converting it into a suitable format.

After preprocessing, the system checks a condition (Kb), which determines how the data will be processed. If the condition is not satisfied, the data is passed to the BERT model, which performs contextual analysis and generates an initial

prediction. Then, the system performs data fetching, where relevant information is collected from external sources, followed by data verification to validate the claim using real-world evidence.

Finally, the system moves to the **decision-making** stage, where both the model prediction and verified evidence are combined to make the final judgment. The result is then displayed as the **final output**, indicating whether the news is real or fake, and the process ends.

3.2.1 Dataset Preparation and Preprocessing

Before training the models, a high-quality dataset is prepared containing labelled news articles and claims. The dataset is carefully cleaned and preprocessed to remove noise and inconsistencies, ensuring better model performance.

Data Collection: Datasets are collected from publicly available fake news datasets and fact-checking sources, ensuring diversity and reliability. Each entry is labelled into categories such as REAL, FAKE, or UNCERTAIN to support multi-class classification.

Data Cleaning: The collected data is preprocessed by removing noise such as HTML tags, special characters, and duplicate entries. Text normalization techniques including lowercasing, tokenization, and stopword removal are applied to make the data suitable for model training.

Data Annotation: Each claim in the dataset is assigned a label based on its authenticity, either manually by experts or using semi-automated methods. This step ensures the quality and correctness of labels, which is crucial for supervised learning.

Data Splitting: The dataset is divided into training, validation, and testing sets to evaluate model performance effectively. This helps in assessing how well the model generalizes to unseen data.

Handling Imbalanced Data: Since fake news datasets are often imbalanced, techniques such as oversampling, undersampling, or class weighting are applied. These methods help improve model fairness and prevent bias toward dominant classes.

Input Formatting: The processed data is formatted according to BERT input requirements, including token IDs, attention masks, and special tokens. This ensures compatibility with transformer-based models.

3.2.2 Text Representation and Classification using BERT

The BERT model is used to extract contextual features from the input text and perform initial classification by understanding the meaning of words in relation to their context. It analyzes the entire sentence bidirectionally to generate meaningful representations, which are then used to predict whether the news is real or fake with an associated probability score.

Input Encoding: The input text is processed using the BERT tokenizer, which converts it into tokens and then into numerical representations such as input IDs, attention masks, and token type IDs. This structured format allows the model to understand sentence boundaries and focus on relevant parts of the input.

Contextual Embedding: BERT processes the input text bidirectionally, meaning it considers both left and right context simultaneously. This helps in capturing deep relationships between words and generating rich contextual embeddings that represent the meaning of the entire sentence.

Feature Learning: Through multiple transformer layers, the model learns semantic and syntactic patterns present in the text. These learned features help distinguish between real and fake news by identifying subtle linguistic cues and contextual differences.

Initial Prediction: A classification head is applied on top of the BERT model to produce an initial probability score (bert_prior). This score indicates the likelihood of the input claim being real or fake based on learned patterns.

Fine-Tuning: The pre-trained BERT model is fine-tuned using the prepared labelled dataset. During this process, model parameters are adjusted to improve performance specifically for fake news detection tasks.

Loss Optimization: Loss functions such as cross-entropy are used to measure the difference between predicted outputs and actual labels. The model minimizes this loss using optimization algorithms like Adam to improve prediction accuracy.

Output Representation: The final output of the model is a probability distribution over the defined classes. The class with the highest probability is selected as the predicted label for the input claim.

3.2.3 Evidence Retrieval using External APIs

To validate claims, the system retrieves real-world evidence from external sources, gathering relevant and up-to-date information to support or refute the given input. This process enhances the accuracy of the system by incorporating factual data beyond model predictions, leading to more reliable decision-making.

Query Generation: The input claim is transformed into a meaningful search query by extracting important keywords and phrases. This ensures that the retrieved results are relevant and closely related to the original claim.

API Integration: The system integrates external APIs such as NewsAPI, Google Search API, and FactCheck APIs to fetch real-time articles and verified information. These APIs provide access to reliable and up-to-date sources for evidence collection.

Data Extraction: Relevant textual content is extracted from the retrieved articles, including headlines, summaries, and key sentences. This step focuses on gathering useful information required for claim verification.

Filtering: Irrelevant, duplicate, or low-quality sources are removed to improve the reliability of the evidence. This ensures that only credible and meaningful information is used for further analysis.

Ranking: The retrieved articles are ranked based on their similarity to the input claim using text similarity measures. Higher-ranked articles are considered more relevant for verification.

Top-N Selection: A fixed number of top-ranked evidence sources (e.g., top 5) are selected for further processing. This helps in reducing computational complexity while maintaining accuracy.

Preparation for NLI: The selected evidence sentences are paired with the original claim to form input for the NLI model. These pairs are used to determine whether the evidence supports or contradicts the claim.

3.2.4 NLI-Based Verification using BART

The BART-based NLI model evaluates the relationship between a given claim and the retrieved evidence by analyzing their semantic meaning. It determines whether the evidence supports (entails), contradicts, or is neutral with respect to the claim, enabling accurate fact verification based on contextual understanding.

Input Pairing: Each input claim is paired with the retrieved evidence sentences to form claim-evidence pairs. These pairs serve as input to the NLI model, allowing it to analyze the logical relationship between the claim and supporting information.

LI Processing: The BART-based model processes each claim-evidence pair to determine the nature of their relationship. It classifies the pair into one of three categories: entailment (support), contradiction (refute), or neutral (no clear relation).

Semantic Understanding: The model captures deep semantic relationships by analyzing context, meaning, and logical connections between the claim and evidence. This enables accurate interpretation beyond simple keyword matching.

Score Assignment: Each claim-evidence pair is assigned a score based on the predicted relationship, indicating the strength of support or contradiction. These scores help quantify the reliability of the evidence.

Aggregation: Scores from multiple evidence sources are combined to form a unified representation of evidence strength. This aggregation helps in making a more balanced and reliable decision.

Confidence Calculation: An overall evidence confidence score is calculated based on aggregated results. This score reflects how strongly the available evidence supports or refutes the claim.

Output Generation: The system produces an evidence-based validation result, which is later combined with the BERT prediction to determine the final classification of the claim.

3.2.5 Hybrid Scoring and Decision Making

The final classification is based on a hybrid scoring mechanism that combines the model's prediction with evidence-based verification scores. This approach ensures a balanced decision by integrating both learned patterns and real-world information, resulting in more accurate and reliable classification of the input.

Score Combination: The system computes a hybrid score by combining the BERT prediction score and the evidence score using the formula: Hybrid Score = (0.45 × BERT Score) + (0.55 × Evidence Score). This ensures that both learned patterns and real-world evidence contribute to the final decision.

Thresholding: The computed hybrid score is compared against predefined threshold values to determine the classification category. These thresholds help in making consistent and reliable decisions.

Multi-Class Classification: Based on the hybrid score, the system classifies the claim into one of four categories: REAL, FAKE, PARTIALLY TRUE, or UNCERTAIN. This allows for more nuanced decision-making compared to simple binary classification.

Uncertainty Handling: If the system does not have sufficient confidence or if the evidence is weak or conflicting, it assigns the label UNCERTAIN. This reduces the risk of incorrect predictions.

Final Output: The system outputs the final classification label along with a confidence score and supporting evidence. This improves transparency and user trust.

Post-processing: The results are formatted in a user-friendly manner, including structured output and readable explanations for better interpretation.

3.2.6 Training Process

The models in TruthLens PRO are trained using labelled datasets consisting of real and fake news samples, enabling them to learn patterns and relationships required for accurate classification. This training process allows the system to understand textual context, detect misinformation, and improve prediction performance over time.

Model Initialization: The BERT and BART models are initialized using pre-trained weights obtained from large-scale datasets. This helps in faster convergence and allows the models to leverage prior linguistic knowledge for better performance.

Training Loop: The models are trained iteratively using batches of training data, where each batch is passed through the network to learn patterns. This iterative process continues over multiple epochs to improve accuracy.

Loss Calculation: A suitable loss function such as cross-entropy is used to measure the difference between predicted outputs and actual labels. Minimizing this loss helps the model make more accurate predictions.

Backpropagation: Gradients are computed based on the loss and propagated backward through the network. This process updates the model parameters to reduce prediction error.

Optimization: Optimization algorithms such as Adam are used to efficiently update model weights. These optimizers help in faster convergence and better performance during training.

Validation: The model's performance is evaluated on a separate validation dataset during training. This helps in monitoring generalization ability and detecting overfitting.

Checkpointing: The best-performing model weights are saved at different stages of training. This ensures that the most optimal model can be used for testing and deployment.

Early Stopping: Training is stopped when there is no significant improvement in validation performance over several epochs. This prevents overfitting and saves computational resources.

3.2.7 Evaluation

The performance of TruthLens PRO is evaluated using standard evaluation metrics to ensure the effectiveness and reliability of the model in detecting fake news. These metrics help in analyzing different aspects of model performance and provide a comprehensive understanding of its accuracy and prediction capability.

Accuracy: Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted instances to the total number of instances. It provides a general idea of model performance.

Precision: Precision measures the proportion of correctly predicted positive instances among all predicted positives. It indicates how reliable the model is when it predicts a claim as fake or real.

Recall: Recall measures the ability of the model to correctly identify all relevant positive instances from the dataset. It reflects how well the model captures actual fake or real news cases.

F1-Score: F1-Score is the harmonic mean of precision and recall, providing a balanced evaluation metric. It is especially useful when dealing with imbalanced datasets.

3.3 Advantages of the Proposed System

The proposed system, TruthLens PRO, offers several significant advantages over traditional fake news detection methods by combining deep learning models with real-time evidence verification.

Contextual Understanding using BERT: The use of BERT enables the system to understand the contextual meaning of text rather than relying on simple keywords. This improves accuracy by capturing semantic relationships and subtle differences in language.

Evidence-Based Verification: Unlike traditional models, the system does not rely solely on predictions. It retrieves real-world evidence using APIs such as NewsAPI and Google Search API, ensuring that decisions are supported by factual information.

Advanced Reasoning with BART (NLI): The BART-based NLI model evaluates whether the retrieved evidence supports or contradicts the claim. This adds a logical reasoning layer, making the system more reliable and trustworthy.

Hybrid Scoring Mechanism: The combination of BERT prediction and evidence score improves overall performance. This reduces false positives and false negatives by balancing learned patterns with real-world validation.

Multi-Class Classification: The system classifies news into multiple categories such as REAL, FAKE, UNCERTAIN, providing more detailed insights compared to binary classification systems.

Real-Time Analysis: By combining deep learning and external verification, the system achieves higher accuracy and reduces the risk of incorrect predictions.

Scalability and Flexibility: The system is scalable and can be extended with additional APIs, datasets, or models, making it adaptable to future improvements and larger datasets.

Reduced Dependency on Manual Features: Unlike traditional feature-based methods, the system automatically learns features from data, reducing manual effort and improving performance.

4 Implementation

To implement the fake news detection system (TruthLens PRO), the initial step involves collecting a diverse and high-quality dataset consisting of real and fake news articles, claims, and fact-checking records. The system integrates advanced Natural Language Processing techniques using transformer-based models such as BERT and BART, along with external APIs for real-time verification.

The implementation focuses on multiple stages including text preprocessing, contextual understanding, evidence retrieval, and hybrid decision-making. During preprocessing, the input text is cleaned, normalized, and tokenized to ensure consistency and improve model performance. The processed text is then passed to BERT, which generates contextual embeddings and performs an initial classification based on semantic understanding.

To enhance reliability, the system retrieves relevant information from external sources such as NewsAPI and Google Search API. These sources provide real-world evidence, which is further analyzed using a BART-based Natural Language Inference (NLI) model to determine whether the evidence supports, contradicts, or is neutral with respect to the claim.

Additionally, the system employs a hybrid scoring mechanism that combines the BERT prediction score with the evidence-based verification score. This approach ensures a balanced decision by considering both learned patterns and factual validation. The final output is generated as a multi-class classification, categorizing the claim into REAL, FAKE, or UNCERTAIN.

By combining deep learning models with real-world evidence sources, the system ensures improved accuracy, robustness, and interpretability. This integrated approach not only reduces false predictions but also enhances user trust by providing transparent and evidence-backed results, making TruthLens PRO a powerful solution for combating misinformation.

Furthermore, the system is designed with a user-friendly web interface developed using Flask, which allows users to easily input news text and receive instant predictions along with supporting evidence. This interface ensures smooth

interaction between the user and the backend models, making the system accessible and practical for real-world usage.

In addition, the architecture of TruthLens PRO is scalable and adaptable, allowing future enhancements such as multilingual support, integration with social media platforms, and real-time deployment. These improvements will further strengthen the system's ability to handle diverse data sources and evolving misinformation patterns, making it more effective in real-world applications.

4.1 Software Requirements

In this project, Python is used as the primary programming language due to its simplicity, flexibility, and extensive support for machine learning and Natural Language Processing libraries. The implementation is carried out using tools such as Anaconda Prompt and Visual Studio Code (VS Code), which provide an efficient development environment.

Anaconda is used for managing packages, dependencies, and virtual environments, making it easier to install and maintain required libraries. The Anaconda Prompt allows smooth execution of Python scripts and environment control.

Visual Studio Code (VS Code) is used as the primary code editor due to its user-friendly interface, debugging support, and extensions for Python development. It enables efficient coding, testing, and project management.

4.2 Hardware Requirements

The implementation of TruthLens PRO requires a system with sufficient computational resources to efficiently train and run deep learning models. A modern computer with a multi-core processor is recommended to handle data preprocessing, model execution, and API interactions smoothly.

A minimum of 8 GB RAM is required to manage datasets and perform model operations, while 16 GB or higher is preferred for better performance, especially during training and handling large text data. Adequate storage space (at least 10–20 GB) is also necessary to store datasets, model files, and project-related resources.

For faster model training and inference, a GPU-enabled system (such as NVIDIA GPU with CUDA support) is highly recommended. However, the system can also run on CPU, although with slower performance. Platforms like Google Colab can be used as an alternative to access free GPU resources.

A stable internet connection is essential for accessing external APIs such as NewsAPI, Google Search API, and FactCheck API, which are used for real-time evidence retrieval. Overall, these hardware requirements ensure smooth development, execution, and deployment of the TruthLens PRO system.

4.3 Libraries and APIs Used

For the proposed system, several libraries and APIs are used to support model training, text processing, and real-time fact verification. These tools help in efficiently handling data, building and training machine learning models, processing and understanding text, and verifying information using up-to-date and reliable sources from the internet.

1. Flask
2. Transformers (HuggingFace)
3. Torch (PyTorch)
4. Scikit-learn
5. Pandas
6. NumPy
7. NewsAPI
8. Google Search API
9. FactCheck API
10. Os

4.3.1 Flask

Flask is a lightweight web framework used to build the frontend interface of the system, enabling seamless interaction between the user and the backend models. It handles user requests, processes input data, and displays the prediction results dynamically, making it suitable for developing simple and efficient web-based applications like TruthLens PRO.

Key features include:

Routing: Routing maps user inputs, such as news text, to the appropriate backend processing functions for prediction, ensuring that requests are handled correctly and efficiently within the system.

Template Engine: The template engine uses Jinja2 to dynamically generate and display prediction results and supporting evidence, providing an interactive and user-friendly interface.

Deployment Support: Flask allows easy integration of machine learning models into a web-based application, enabling smooth deployment and real-time access to the system's functionalities.

4.3.2 Transformers (HuggingFace)

The Transformers library provides access to powerful pre-trained models such as BERT and BART, which are essential for natural language processing tasks in this project. It enables efficient implementation of text understanding, classification, and inference by leveraging state-of-the-art architectures, thereby improving the overall performance and accuracy of the TruthLens PRO system.

Key features include:

Pre-trained Models: Provides ready-to-use models such as BERT and BART that are already trained on large-scale datasets, enabling faster development and improved performance without training from scratch.

Tokenization: Converts raw input text into tokens that the model can understand, preparing the data in a structured format suitable for processing by transformer-based models.

Fine-tuning: Allows customization of pre-trained models on specific fake news datasets, helping improve accuracy and adapt the models to the requirements of the TruthLens PRO system.

4.3.3 Torch (PyTorch)

PyTorch is used as the core deep learning framework for implementing and training transformer models in this project. It provides a flexible and dynamic environment for

building neural networks, supports automatic differentiation, and enables efficient model training and experimentation, making it well-suited for developing advanced NLP systems like TruthLens PRO.

Key features include:

Dynamic Computation Graph: Enables flexible model design and experimentation by allowing the structure of the neural network to be modified dynamically during runtime, making it easier to develop and debug models.

Automatic Differentiation: Automatically computes gradients during training, simplifying the backpropagation process and reducing the need for manual calculations in model optimization.

GPU Support: Accelerates training and inference by utilizing CUDA-enabled GPUs, significantly improving performance when working with large datasets and complex models.

4.3.4 Scikit-learn

Scikit-learn is used for preprocessing, evaluation, and auxiliary machine learning tasks in this project. It provides efficient tools for data cleaning, feature transformation, and performance evaluation using metrics such as accuracy, precision, recall, and F1-score, supporting the overall effectiveness of the TruthLens PRO system.

Key features include:

Data Splitting: Data splitting means dividing your dataset into three parts: training, validation, and testing sets. The training set is used to teach the model, the validation set is used to tune and improve the model, and the testing set is used to check how well the model performs on new, unseen data.

Evaluation Metrics: Evaluation metrics are used to measure how good a model is. Common metrics include accuracy (how often the model is correct), precision (how many predicted positives are actually correct), recall (how many actual positives the model correctly finds), and F1-score (a balance between precision and recall).

Feature Utilities: Feature utilities help prepare and improve the data before using it in a model. This includes tasks like scaling values, encoding categories into numbers, handling missing data, and other preprocessing steps that make the data easier for the model to understand.

4.3.5 Pandas

Pandas is used for handling and manipulating structured datasets in this project. It provides powerful data structures such as DataFrames for organizing, cleaning, and analyzing data efficiently, enabling smooth preprocessing and preparation of datasets for the TruthLens PRO system.

Key features include:

Data Handling: Data handling means efficiently loading and managing large datasets so they can be used easily for analysis or machine learning. It ensures the data is organized and accessible without slowing down the system.

Data Cleaning: Data cleaning involves fixing or removing bad data such as duplicates, missing values (nulls), and incorrect or inconsistent entries. This helps improve the quality and accuracy of the dataset.

Data Transformation: Data transformation means converting raw data into a proper structured format that can be used for training models. This may include changing formats, encoding data, or organizing it into useful features.

4.3.6 NumPy

NumPy is used for numerical computations and efficient data representation in this project. It provides support for multi-dimensional arrays and mathematical operations, enabling fast processing and transformation of data required for model training and evaluation in the TruthLens PRO system.

Key features include:

Array Operations: Array operations refer to efficiently handling and manipulating numerical data stored in arrays. This includes tasks like adding, reshaping, slicing, and filtering data, which makes working with large datasets easier and faster.

Mathematical Functions: Mathematical functions provide essential calculations needed during model training. These include operations like mean, sum, exponentials, and gradients, which help in analyzing data and improving machine learning models.

Performance Optimization: Performance optimization focuses on making data processing faster and more efficient. It reduces computation time and improves resource usage so large datasets and complex models can run smoothly.

4.3.7 NewsAPI

NewsAPI is used to fetch real-time news articles related to the input claim in this project. It enables the system to retrieve up-to-date and relevant information from multiple trusted sources, which is then used for evidence-based verification to improve the accuracy and reliability of the TruthLens PRO system.

Key features include:

Real-Time Data: Real-time data means collecting the latest and most up-to-date information from multiple sources. This helps ensure that the system always works with current news and events.

Evidence Collection: Evidence collection involves gathering information that supports or contradicts a given claim or topic. This helps in verifying facts and improving the quality of answers.

Reliability: Reliability means improving the trustworthiness of the system by using real-world and verified data. This makes the results more accurate and dependable.

4.3.8 Google Search API

Google Search API is used to retrieve relevant web results for fact-checking in this project. It allows the system to access a wide range of online sources, providing additional context and supporting evidence that enhances the verification process and improves the overall reliability of the TruthLens PRO system.

Key features include:

Wide Coverage: Wide coverage means the system can access information from many different sources across the internet. This helps gather diverse viewpoints and more complete information.

Query-Based Search: Query-based search means the system finds results based on the specific input or claim given by the user. It focuses on relevant information that matches the question or topic.

Evidence Support: Evidence support means using information from multiple sources to confirm or validate a claim. This helps improve accuracy and reduces the chances of incorrect answers.

4.3.9 FactCheck API

FactCheck APIs provide access to verified claims and fact-checking results from trusted sources in this project. They help validate the authenticity of information by supplying reliable evidence, thereby strengthening the accuracy and credibility of the TruthLens PRO system.

Key features include:

Verified Data: Verified data means collecting information that has already been fact-checked by trusted organizations. This ensures the system uses reliable and accurate information.

Claim Matching: Claim matching involves comparing a user's input or statement with existing fact-check records. This helps identify whether the claim is true, false, or partially true.

Accuracy Improvement: Accuracy improvement means reducing incorrect predictions by relying on trusted and verified sources. This makes the system more dependable and precise.

4.3.10 OS

The os module in Python is used to interact with the operating system and manage files and directories efficiently in TruthLens PRO. It enables operations such as handling file paths, organizing datasets, accessing stored models, and managing system-level tasks, ensuring smooth execution and proper file management throughout the project.

Key features include:

File and Directory Operations: File and directory operations involve creating, deleting, and managing files, datasets, and model outputs. This helps organize data properly and makes it easy to store and access information when needed.

Path Manipulation: Path manipulation means handling file paths in a way that works across different operating systems. It ensures files can be located and accessed correctly regardless of the platform being used.

Platform Information: Platform information provides details about the system being used, such as the operating system and environment. This helps ensure the software is properly configured and compatible with the system.

4.4 Code

Before implementing our code, we understand that the current workflow or proposed system needs to be divided into four modules for proper development. These modules help organize the system into smaller, manageable parts, making it easier to design, implement, and test each functionality step by step.

1. Flask application and routes
2. Model definition
3. Utility functions
4. Text Processing and Evidence Retrieval Functions

Let's delve deeper into each module:

Flask Application and Routes: The Flask application serves as the core interface of the TruthLens PRO system, enabling seamless interaction between users and the backend models. It acts as a bridge that connects the frontend user interface with the underlying machine learning components, ensuring smooth communication and processing.

The application handles HTTP requests such as GET and POST, allowing users to submit news text or claims through web forms or API calls. It processes the

input data and forwards it to the appropriate backend modules for analysis and prediction.

Once the input is processed, the Flask application generates outputs in different formats based on the request type. It renders dynamic HTML pages to display prediction results in a user-friendly interface, making it easy for users to understand the outcome.

In addition, it can return JSON responses containing the classification label, confidence score, and retrieved supporting evidence. This ensures flexibility for both web-based interaction and API-based integration, enhancing the usability and functionality of the system.

Model Definition (BERT + BART): This module is responsible for defining and integrating the deep learning models used in the TruthLens PRO system. It includes the BERT model for text classification and the BART-based NLI model for evidence verification, enabling the system to perform both contextual understanding and factual validation of news content.

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based deep learning model developed by Google for natural language processing tasks. It processes text in both directions, allowing it to understand the context and relationships between words more effectively, which helps in accurately classifying news as real or fake.

BART (Bidirectional and Auto-Regressive Transformers) is a transformer-based model developed by Facebook AI that combines bidirectional encoding and autoregressive decoding. It is used for natural language inference by analyzing the relationship between a claim and its supporting evidence, determining whether the evidence entails, contradicts, or is neutral with respect to the claim.

The inputs to this module include preprocessed textual data such as news articles or claims, along with model configurations like hyperparameters and pretrained weights. These inputs are used to initialize and fine-tune the models for the specific task of fake news detection.

The outputs of this module include the complete model architecture, consisting of the BERT model for contextual text classification and the BART model for natural language inference, along with classification layers and scoring mechanisms. Additionally, it produces trained model weights, including fine-tuned BERT weights for fake news classification and BART weights for evidence validation, which are used during prediction and deployment.

Utility Functions: This module consists of utility functions that provide reusable components for performing common operations within the TruthLens PRO system. These functions help simplify the overall workflow by handling repetitive tasks such as text preprocessing, API interaction, data formatting, and error handling, thereby improving code efficiency and modularity.

The inputs to this module include raw text data entered by the user, responses retrieved from external APIs, and various configuration parameters required for processing. These inputs are handled and transformed by the utility functions to ensure consistency and proper formatting for further stages of the system.

The outputs of this module include cleaned and formatted text data that is ready for model processing, along with processed API results where relevant information is extracted and ranked as supporting evidence. Additionally, the module generates logs, validation messages, and error handling outputs, which help in debugging, monitoring system performance, and ensuring smooth execution.

Text Processing and Evidence Retrieval Functions: This module is responsible for handling text preprocessing, feature preparation, and retrieval of real-world evidence using external APIs in the TruthLens PRO system. It plays a crucial role in combining machine learning predictions with factual verification, ensuring that the system produces accurate and reliable results.

It ensures that the input data is clean, structured, and suitable for model processing by performing tasks such as text cleaning, normalization, and tokenization. This preprocessing step prepares the data for effective analysis by the BERT model.

In addition to preprocessing, the module retrieves relevant information from external sources such as NewsAPI, Google Search API, and FactCheck sources. This

helps gather real-world evidence that supports or contradicts the input claim, improving the credibility of the system's predictions.

The inputs to this module include raw news text or claims provided by the user, along with external API data obtained from various sources. These inputs are processed to extract meaningful information required for further analysis.

The outputs include preprocessed and tokenized text ready for BERT input, retrieved and filtered evidence articles, and ranked evidence pairs prepared for NLI processing. Finally, the processed data is passed to the BART model for verification, ensuring accurate and evidence-based classification.

5 Results

The fake news detection system follows a structured and systematic workflow that combines deep learning models with real-world evidence to produce accurate and reliable results. Each step in the process is designed to progressively analyze, verify, and classify the input claim, ensuring both contextual understanding and factual validation.

User Inputs Claim: The user accesses the web application and enters a news claim or statement in the input field. The system accepts text input through the Flask-based interface for verification.

Text Preprocessing: The input claim is cleaned and preprocessed by removing unnecessary characters and formatting it into a structured format suitable for model input. Tokenization is applied to prepare the text for BERT processing.

Initial Classification using BERT: The processed text is passed through the BERT model, which generates contextual embeddings and provides an initial probability score indicating whether the claim is likely real or fake.

Evidence Retrieval: The system generates a query from the input claim and retrieves relevant articles using external APIs such as NewsAPI and Google Search API. These articles serve as supporting evidence for verification.

NLI-based Verification using BART: The retrieved evidence is paired with the input claim and processed using the BART-based Natural Language Inference model. The model determines whether the evidence supports, contradicts, or is neutral to the claim.

Hybrid Score Calculation: The system combines the BERT score and evidence score using a weighted hybrid approach. This improves the overall reliability and accuracy of the final prediction.

Final Classification: Based on the hybrid score, the system classifies the claim into categories such as REAL, FAKE and UNCERTAIN.

Result Visualization: The final result is displayed on the web interface along with supporting statistics such as number of supporting and contradicting evidences. The output includes the classification label, confidence score, and explanation.

After successful running the code:

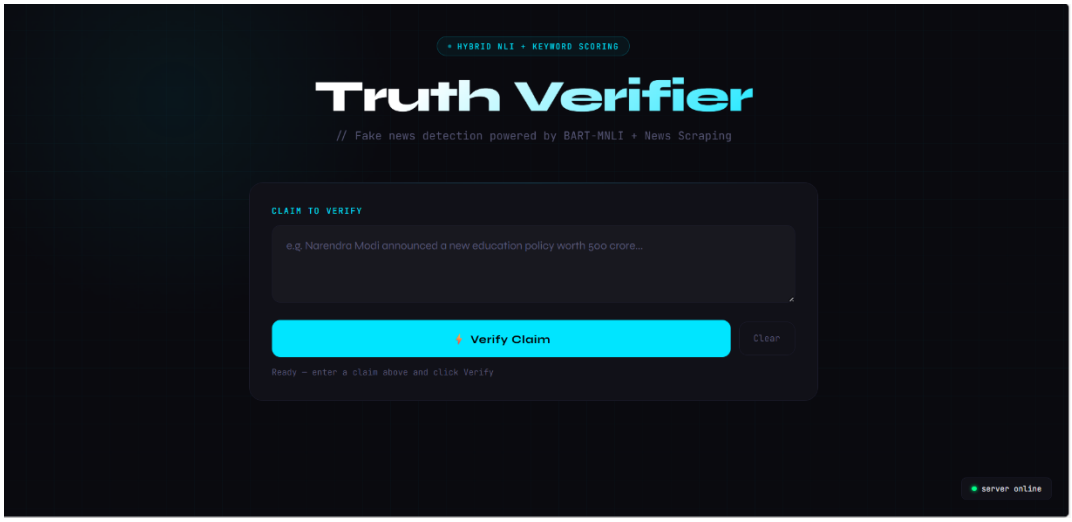
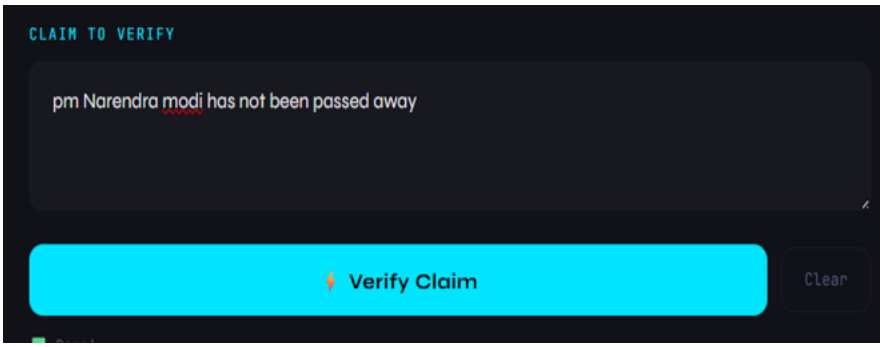
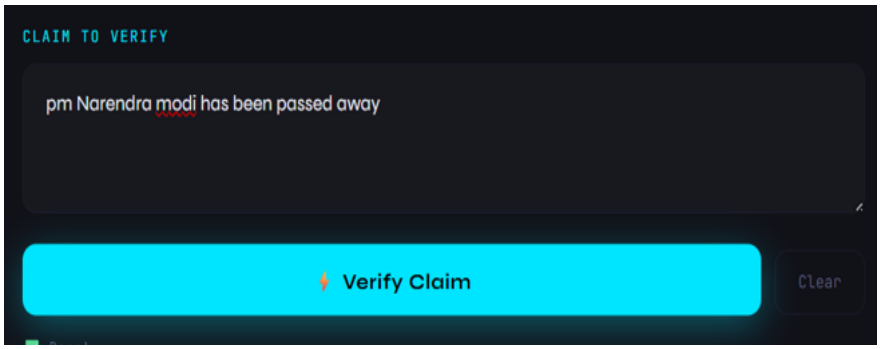
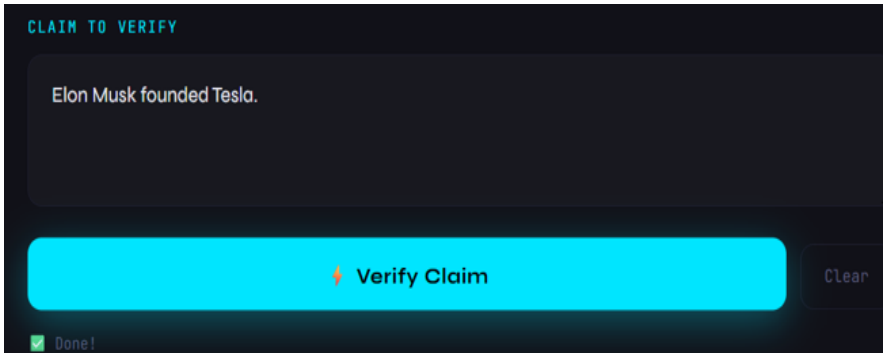
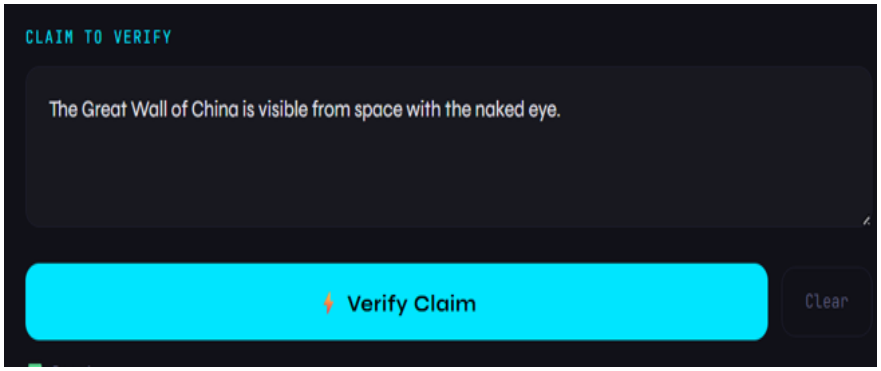
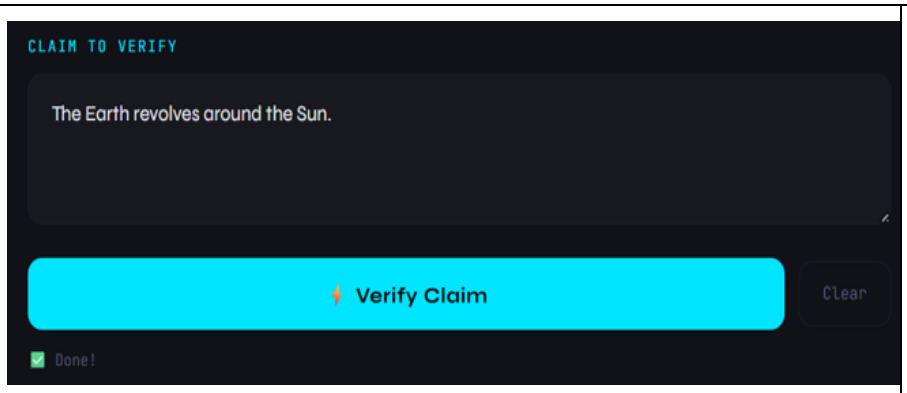
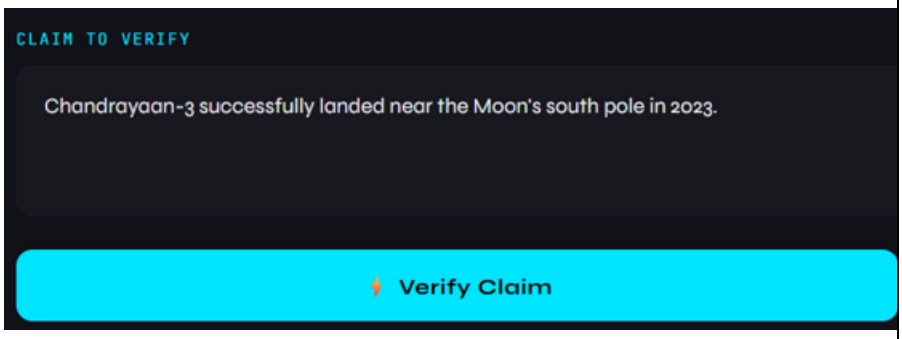
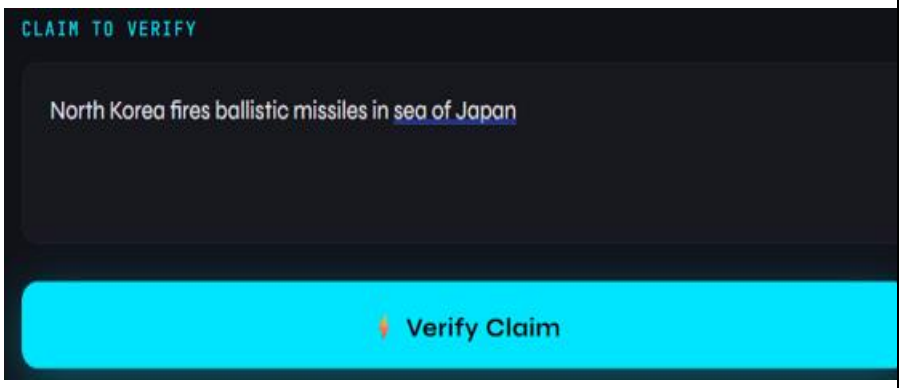


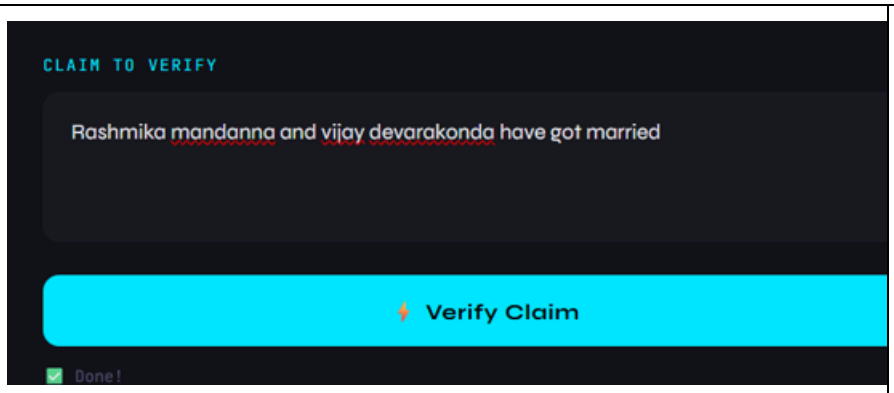
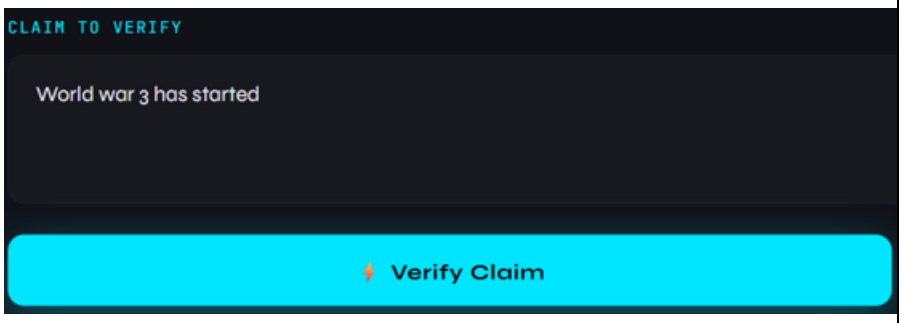
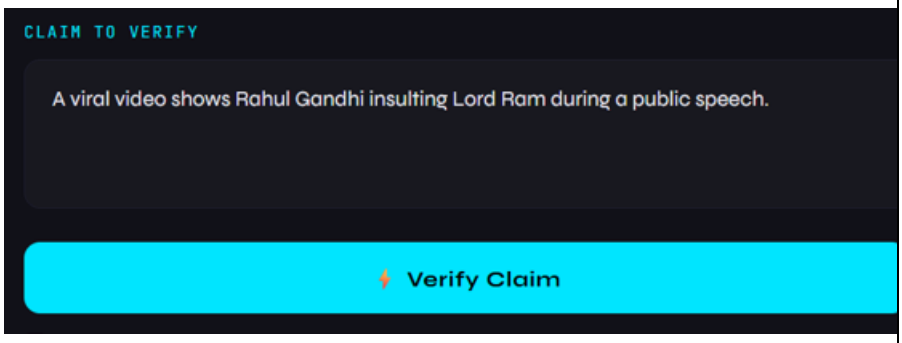
Figure 5.1 User input

After entering the news prompt or claim, the user clicks the "Verify Claim" button, which triggers the system to process the input through various stages including preprocessing, model analysis, and evidence verification. Once the processing is complete, the system displays the result on the screen, showing the final classification along with confidence score and supporting evidence as illustrated above.

These are the outputs that are predicted and the result is displayed as mentioned in the below table.

S.no	Input News/Prompt	Predicted class
1		Real
2		Fake
3		Fake
4		Fake

5		**Real**
6		**Uncertain**
7		**Real**

8		**Real**
9		**Uncertain**
10		**Fake**

6 Conclusion

In conclusion, the proposed system TruthLens PRO demonstrates an effective approach to fake news detection by leveraging advanced Natural Language Processing techniques such as BERT and BART-based Natural Language Inference. The use of contextual embeddings through BERT enables accurate understanding of the semantic meaning of claims, while the BART NLI model enhances verification by analyzing relationships between claims and real-world evidence.

The integration of external sources such as NewsAPI and Google Search API plays a crucial role in strengthening the system's reliability. By retrieving real-time evidence and validating claims against trusted information, the system moves beyond traditional text classification and provides a more explainable and evidence-based decision-making process.

The hybrid scoring mechanism further improves performance by combining model-based predictions with evidence-based validation. This approach allows the system to classify news into multiple categories such as REAL, FAKE and UNCERTAIN, making it more robust and practical for real-world applications.

Although the system shows promising results, certain challenges remain, such as handling ambiguous claims, dependency on external API responses, and limitations in detecting very recent or low-resource information. Future improvements can focus on enhancing evidence retrieval, incorporating multilingual support, and optimizing model efficiency for large-scale deployment.

Overall, the proposed system provides a reliable, scalable, and intelligent solution for fake news detection, contributing to the ongoing efforts to combat misinformation and promote trustworthy information in the digital era.

Furthermore, the system enhances transparency by providing supporting evidence along with the final prediction, allowing users to understand how the decision was made. This improves user trust and makes the system more explainable compared to traditional black-box models that only provide a classification result without justification.

The modular design of TruthLens PRO also ensures flexibility and ease of maintenance. Each component, including preprocessing, model inference, evidence retrieval, and scoring, can be independently updated or improved without affecting the entire system. This makes it easier to incorporate future advancements in machine learning and natural language processing.

In addition, the system is designed to be user-friendly and accessible through a web-based interface, enabling users to verify claims quickly and efficiently. This practical implementation ensures that the system can be used in real-world scenarios by individuals, organizations, and researchers.

As misinformation continues to evolve, systems like TruthLens PRO play a critical role in addressing the challenges of digital information credibility. By combining deep learning with real-world verification, the proposed system not only improves detection accuracy but also contributes to building a more informed and trustworthy digital environment.

7 Future Work

Although the proposed system achieves reliable results in fake news detection, several enhancements can be considered for future development. One important direction is improving the evidence retrieval mechanism by integrating more advanced and diverse fact-checking sources to ensure higher accuracy and broader coverage. This would help the system cross-verify information more effectively and reduce incorrect classifications caused by limited data sources.

Another significant improvement is incorporating multilingual support, enabling the system to analyze and verify news in multiple languages. This would greatly enhance its applicability in real-world scenarios, especially in countries with diverse linguistic populations, and make the system more globally usable.

Additionally, optimizing the system for real-time processing and reducing dependency on external APIs can significantly improve performance, response time, and system stability. Efficient caching mechanisms and lightweight models can also be introduced to ensure faster predictions without compromising accuracy.

Future work can further explore multimodal fake news detection by including images, videos, and audio along with text. Since misinformation is often spread through multiple formats, analyzing only text may not be sufficient. Combining different data types will make the system more robust and reliable.

Moreover, implementing explainable AI (XAI) techniques can enhance user trust by clearly showing why a prediction is classified as real, fake, or uncertain. Providing transparent reasoning and highlighting supporting evidence will make the system more interpretable and user-friendly.

Finally, continuous model updating and training using recent and real-world datasets is essential to keep the system up to date with evolving misinformation trends. Incorporating user feedback loops and adaptive learning techniques can further improve long-term accuracy and system performance.

Another potential enhancement is the integration of social media analysis to track the spread and credibility of information across platforms. By analyzing user engagement, sharing patterns, and source credibility, the system can gain additional context about how information propagates, helping to identify suspicious or viral misinformation more effectively.

Furthermore, deploying the system as a browser extension or mobile application can significantly improve accessibility and usability. This would allow users to verify news in real time while browsing or using social media platforms, making TruthLens PRO more practical and impactful in everyday digital interactions.

8 Bibliography

[1] S. Kumar and N. Shah, "False Information on Web and Social Media: A Survey," *ACM SIGKDD Explorations Newsletter*, vol. 20, no. 2, pp. 1–19, 2018.

[2] H. Nguyen, F. Fang, J. Yamagishi, and I. Echizen, "Multi-task Learning for Detecting and Segmenting Manipulated Facial Images and Videos," in *Proc. IEEE Int. Conf. on Biometrics (ICB)*, 2019, pp. 1–8.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] M. Lewis *et al*., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proc. ACL*, 2020, pp. 7871–7880.

[5] D. D. Lewis and W. A. Gale, "A Sequential Algorithm for Training Text Classifiers," in *Proc. ACM SIGIR Conf.*, 1994, pp. 3–12.

[6] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake News Detection on Social Media: A Data Mining Perspective," *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.

[7] NewsAPI, "News API Documentation," [Online]. Available: https://newsapi.org/.

[8] Google Developers, "Custom Search JSON API," [Online]. Available: https://developers.google.com/custom-search/v1/overview.

[9] S. Vosoughi, D. Roy, and S. Aral, "The spread of true and false news online," *Science*, vol. 359, no. 6380, pp. 1146–1151, 2018.

[10] Z. Zhang, Y. Zhao, and Q. Yang, “Text classification based on convolutional neural networks,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 1487–1498, 2020.

[11] A. Vaswani et al., “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[12] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv preprint arXiv:1907.11692, 2019.

[13] T. Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” arXiv preprint arXiv:1301.3781, 2013.

[14] S. Wang, G. Li, and J. Huang, “Fake News Detection Using Deep Learning Methods,” *IEEE Access*, vol. 9, pp. 168432–168444, 2021.

[15] J. Pennington, R. Socher, and C. Manning, “GloVe: Global Vectors for Word Representation,” in *EMNLP*, 2014, pp. 1532–1543.